



# Today's Class

---

- Text Clustering: Background
- Text Clustering Algorithms
  - Partitional clustering
  - Hierarchical clustering
- Text Clustering with Deep Neural Network
- Topic Model

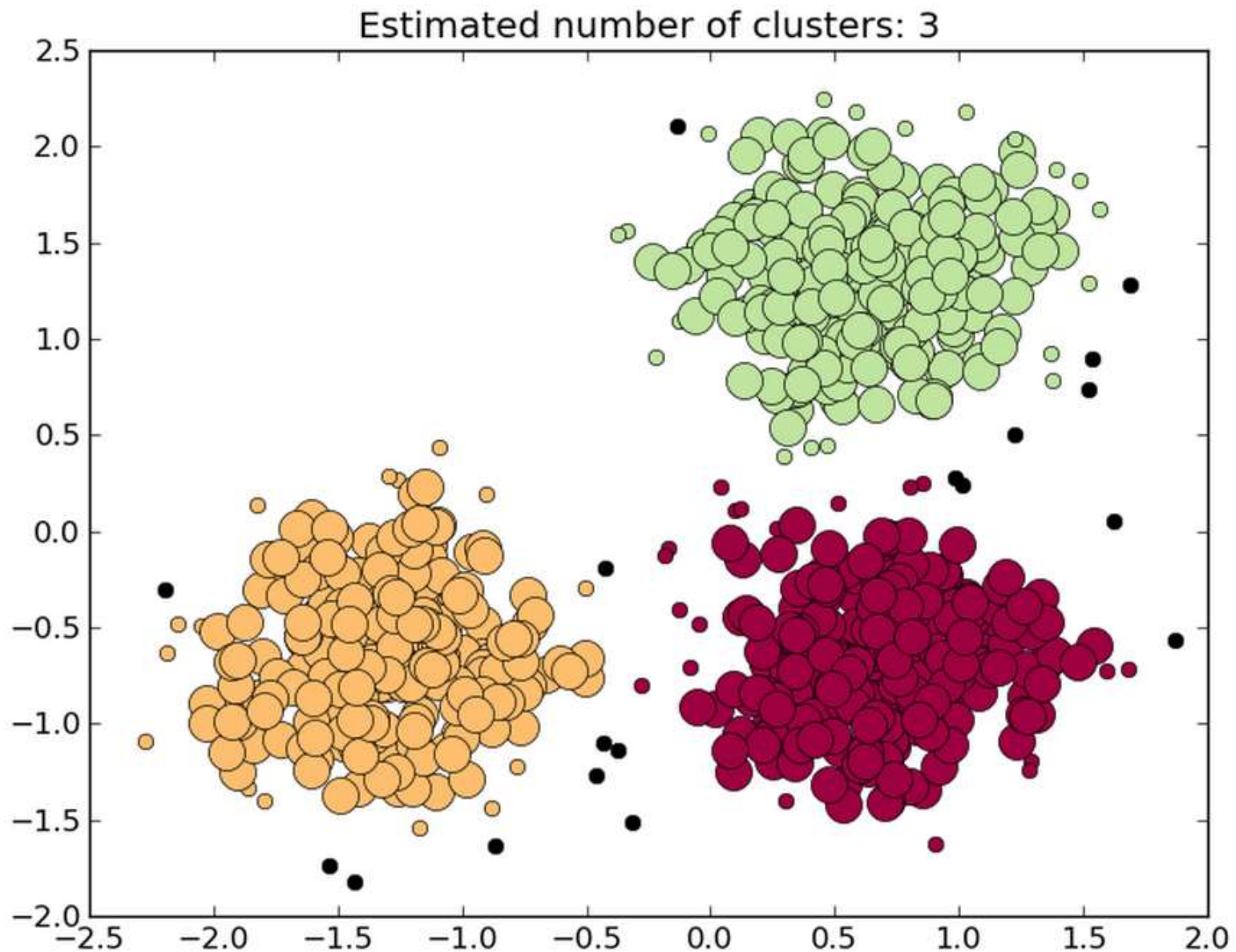
# Clustering

---

- Partition unlabeled examples into disjoint subsets of *clusters*, such that:
  - Examples within a cluster are very similar: high intra-cluster similarity
  - Examples in different clusters are very different: low inter-cluster similarity
- Informally, finding natural groupings among objects.
- Discover new categories in an *unsupervised* manner (no sample category labels provided).

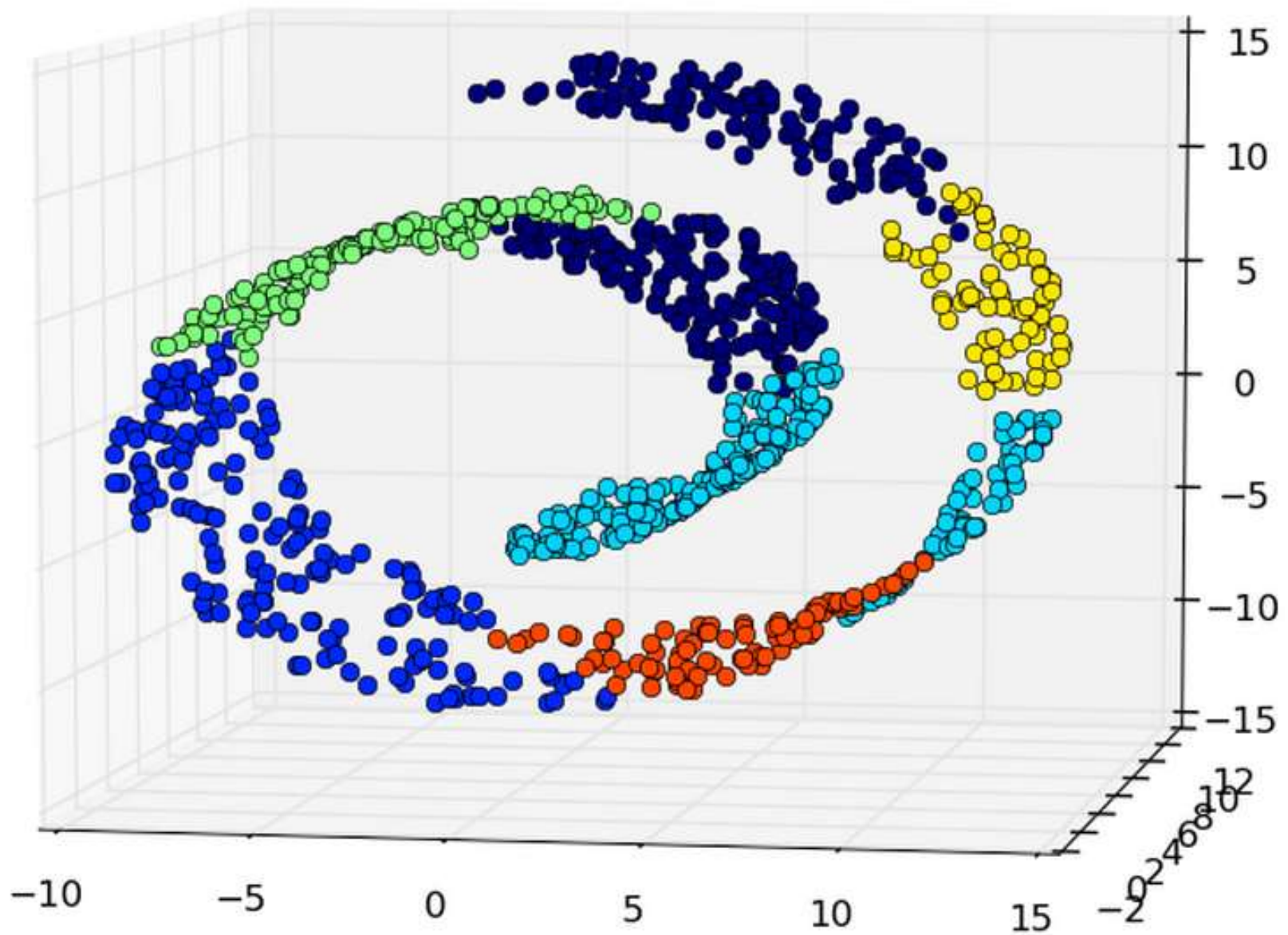
# Clustering Examples

---

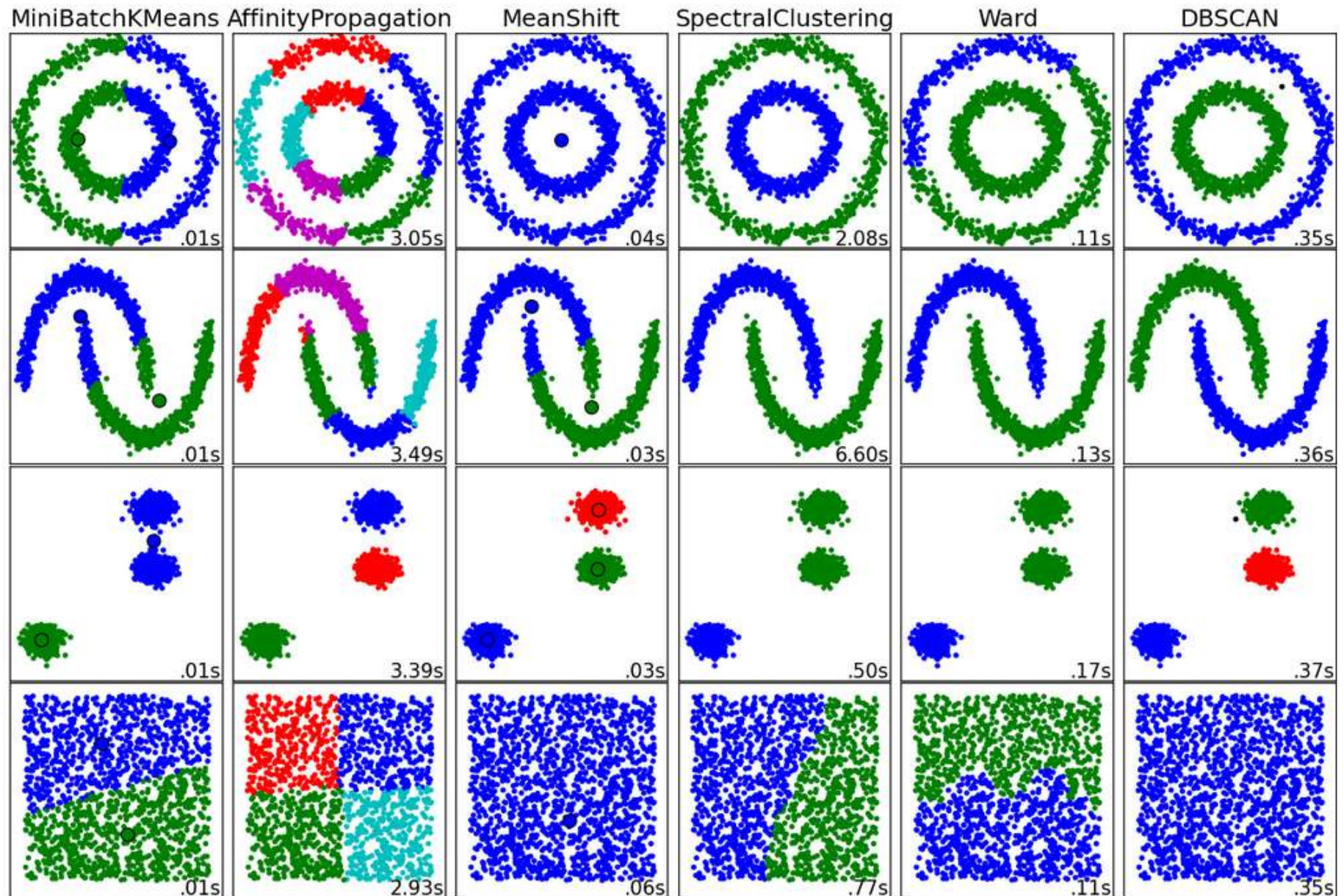


# Clustering Examples

---



# Comparing different clustering algorithms on toy datasets



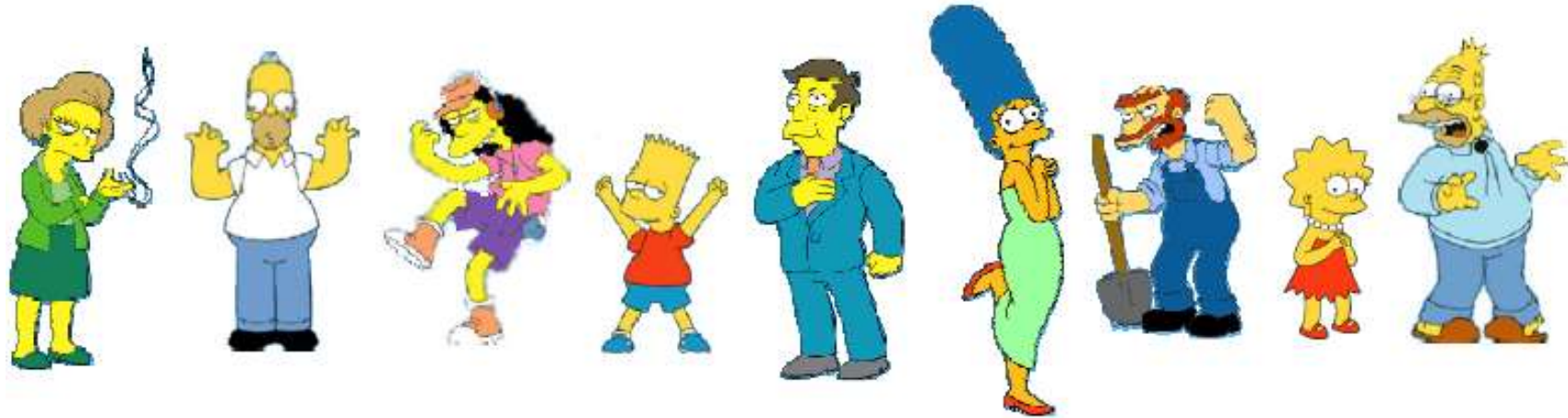
# Why clustering?

---

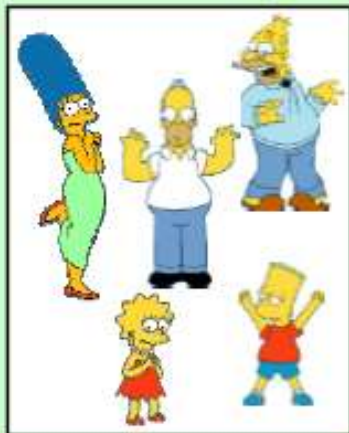
- Organizing data into clusters shows internal structure of the data  
Ex. Clusty and clustering genes above
- Sometimes the partitioning is the goal  
Ex. Market segmentation
- Prepare for other AI techniques  
Ex. Summarize news (cluster and then find centroid)
- Techniques for clustering is useful in knowledge discovery in data  
Ex. Underlying rules, reoccurring patterns,

# What is natural grouping among these objects

---



Clustering is subjective



Simpson's Family



School Employees



Females



Males

# Desirable Properties of a Clustering Algorithm

---

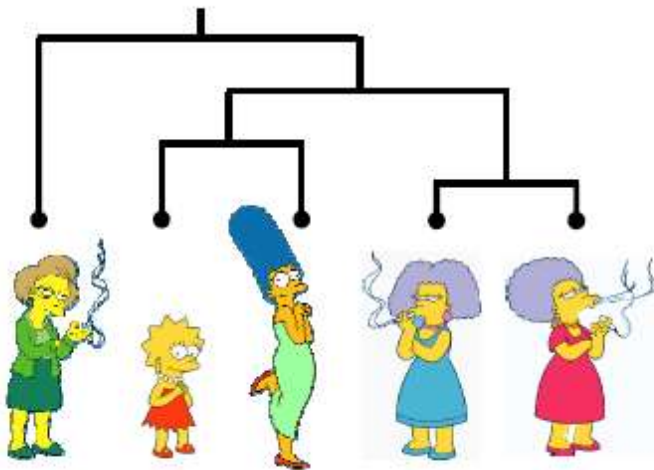
- Scalability (in terms of both time and space)
- Ability to deal with different data types
- Minimal requirements for domain knowledge to determine input parameters
- Able to deal with noise and outliers
- Insensitive to order of input records
- Incorporation of user-specified constraints
- Interpretability and usability

# Two Types of Clustering

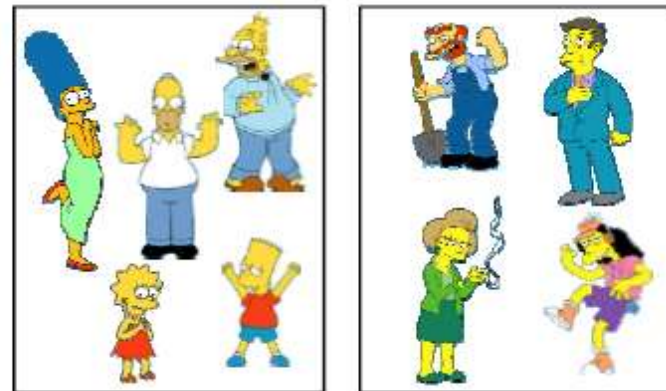
---

- Hierarchical algorithms: Create a hierarchical decomposition of the set of objects using some criterion
- Partitional algorithms: Construct various partitions and then evaluate them by some criterion

**Hierarchical** Agglomerative



**Partitional** Divisive



# Aglomerative vs. Divisive Clustering

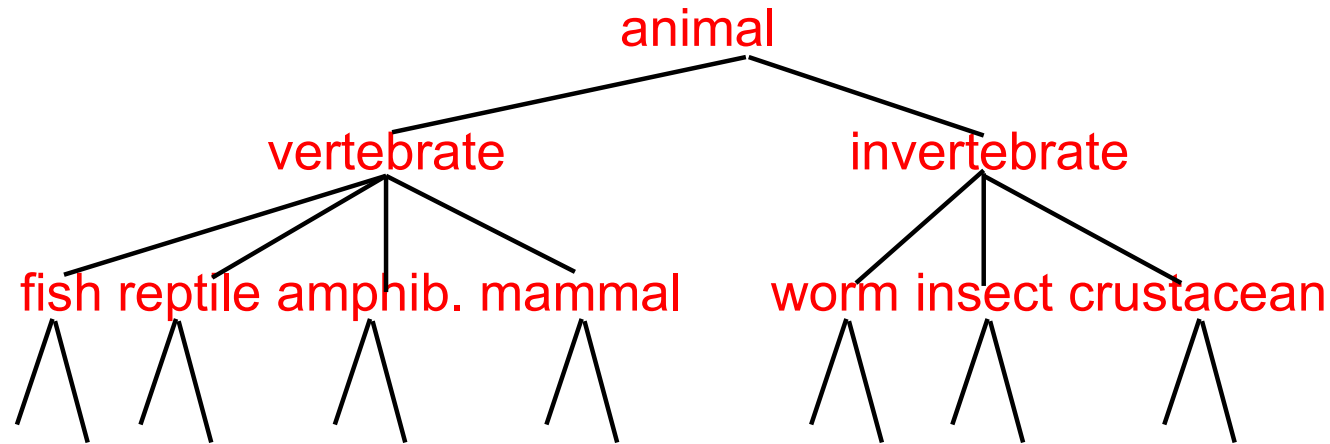
---

- *Agglomerative* (*bottom-up*) methods start with each example in its own cluster and iteratively combine them to form larger and larger clusters.
- *Divisive* (*partitional, top-down*) separate all examples immediately into clusters.

# Hierarchical Clustering

---

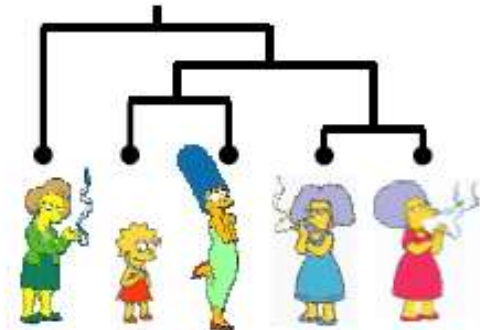
- Build a tree-based hierarchical taxonomy (*dendrogram*) from a set of unlabeled examples.



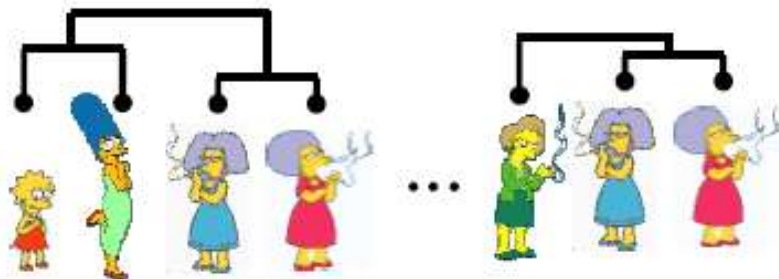
- Recursive application of a standard clustering algorithm can produce a hierarchical clustering.

## Bottom-Up (agglomerative):

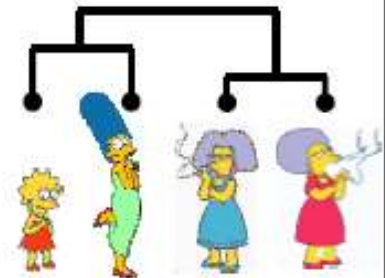
Starting with each item in its own cluster, find the best pair to merge into a new cluster. Repeat until all clusters are fused together.



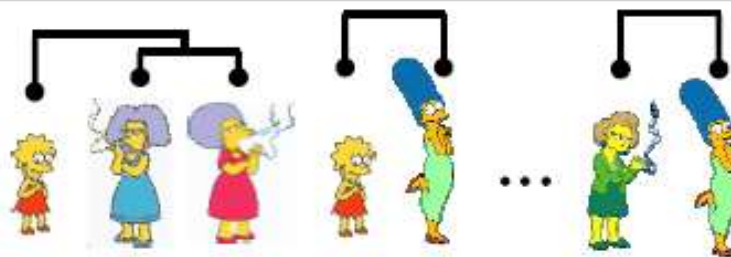
Consider all possible merges...



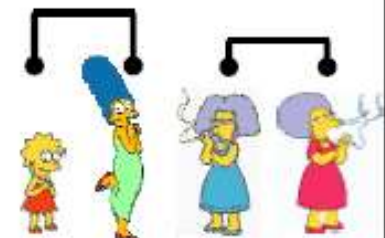
Choose the best



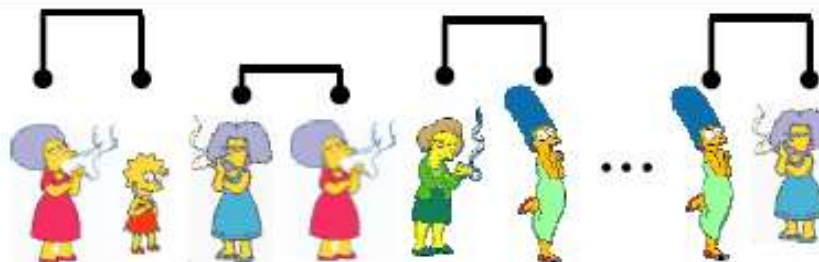
Consider all possible merges...



Choose the best



Consider all possible merges...



Choose the best



# Hierarchical Agglomerative Clustering (HAC)

---

- Assumes a *similarity function* for determining the similarity of two instances.
- Starts with all instances in a separate cluster and then repeatedly joins the two clusters that are most similar until there is only one cluster.
- The history of merging forms a binary tree or hierarchy.

# HAC Algorithm

---

Start with all instances in their own cluster.

Until there is only one cluster:

    Among the current clusters, determine the two clusters,  $c_i$  and  $c_j$ , that are most similar.

    Replace  $c_i$  and  $c_j$  with a single cluster  $c_i \cup c_j$

# Cluster Similarity

---

- Assume a similarity function that determines the similarity of two instances:  $sim(x,y)$ .
  - Cosine similarity of document vectors.
- How to compute similarity of two clusters each possibly containing multiple instances?
  - **Single Link**: Similarity of two most similar members.
  - **Complete Link**: Similarity of two least similar members.
  - **Group Average**: Average similarity between members.

# Single Link Agglomerative Clustering

---

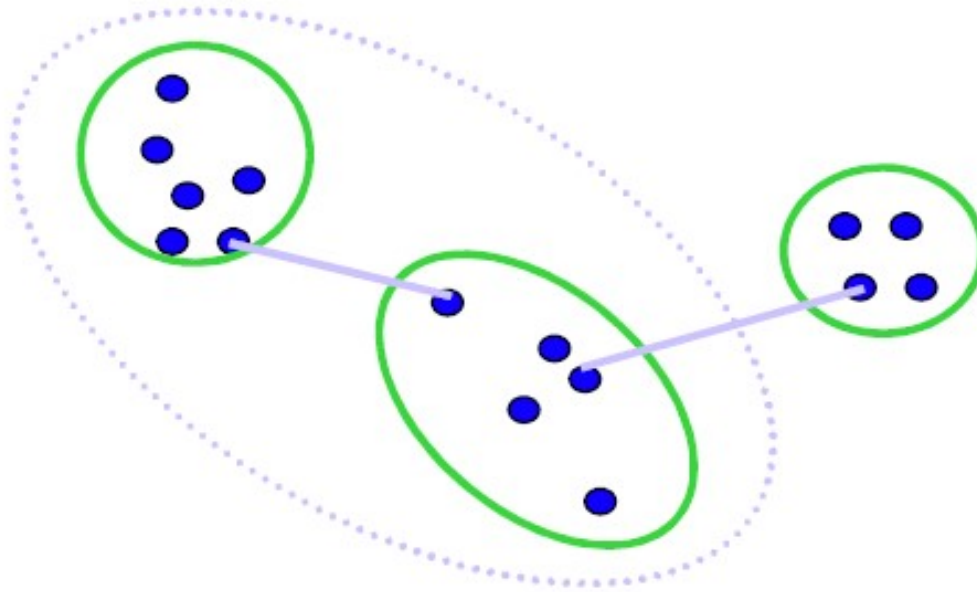
- Use maximum similarity of pairs:

$$\text{sim}(c_i, c_j) = \min_{x \in c_i, y \in c_j} \text{sim}(x, y)$$

- Can result in “straggly” (long and thin) clusters due to *chaining effect*.
  - Appropriate in some domains, such as clustering islands.

# Single Link Example

---



# Complete Link Agglomerative Clustering

---

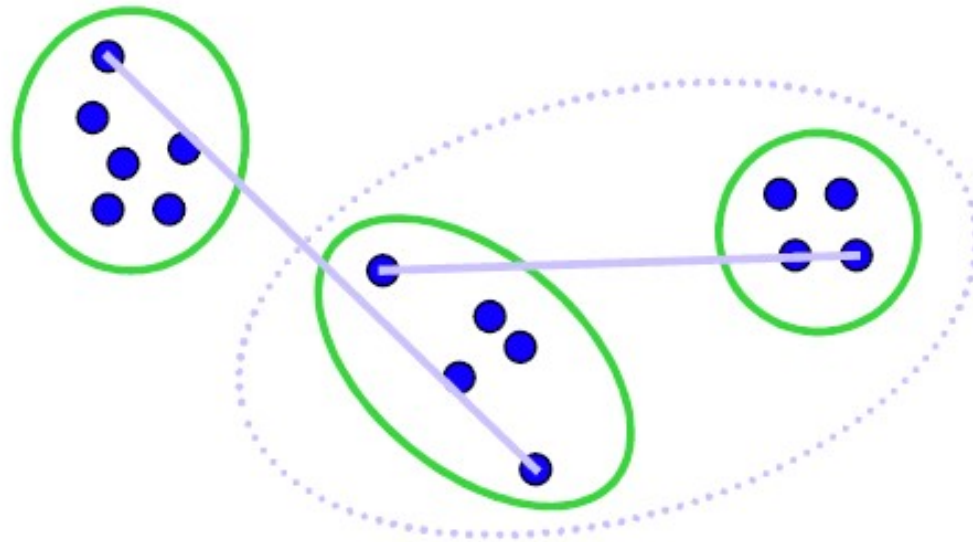
- Use minimum similarity of pairs:

$$\textit{sim}(c_i, c_j) = \max_{x \in c_i, y \in c_j} \textit{sim}(x, y)$$

- Makes more “tight,” spherical clusters that are typically preferable.

# Complete Link Example

---



# Computing Cluster Similarity

---

- After merging  $c_i$  and  $c_j$ , the similarity of the resulting cluster to any other cluster,  $c_k$ , can be computed by:

- Single Link:

$$\text{sim}((c_i \cup c_j), c_k) = \min(\text{sim}(c_i, c_k), \text{sim}(c_j, c_k))$$

- Complete Link:

$$\text{sim}((c_i \cup c_j), c_k) = \max(\text{sim}(c_i, c_k), \text{sim}(c_j, c_k))$$

# Computational Complexity

---

- In the first iteration, all HAC methods need to compute similarity of all pairs of  $n$  individual instances which is  $O(n^2)$ .
- In each of the subsequent  $n-2$  merging iterations, it must compute the distance between the most recently created cluster and all other existing clusters.
- In order to maintain an overall  $O(n^2)$  performance, computing similarity to each other cluster must be done in constant time.

# Group Average Agglomerative Clustering

---

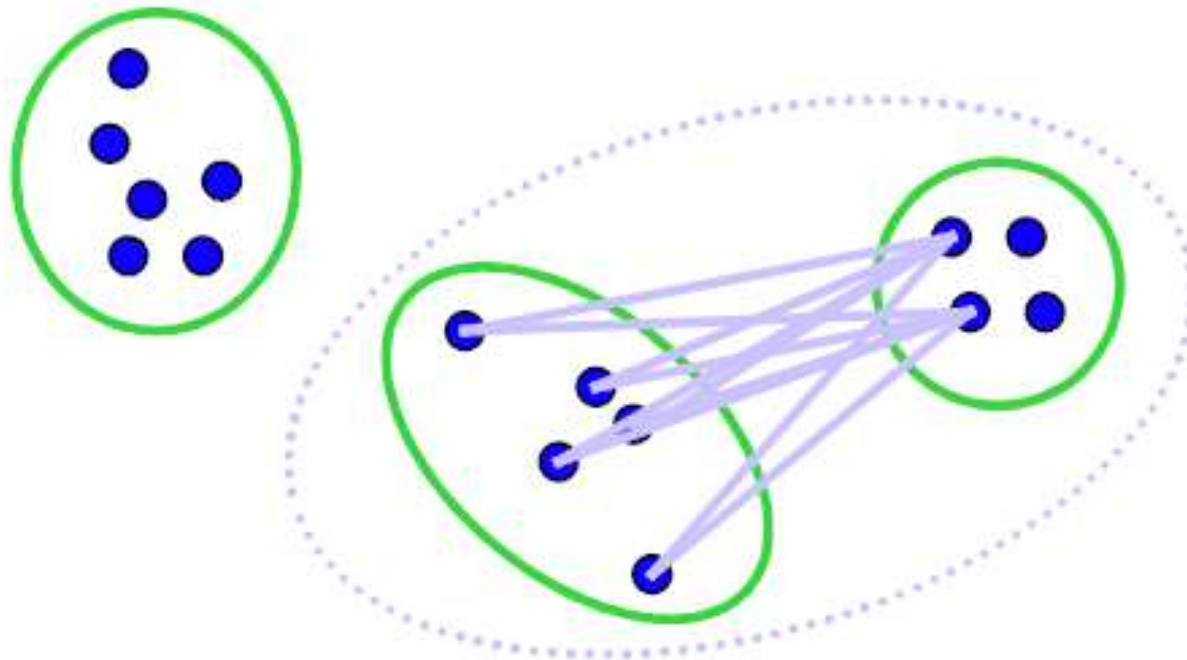
- Use average similarity across all pairs within the merged cluster to measure the similarity of two clusters.

$$\text{sim}(c_i, c_j) = \frac{1}{|c_i \cup c_j|(|c_i \cup c_j| - 1)} \sum_{\vec{x} \in (c_i \cup c_j)} \sum_{\vec{y} \in (c_i \cup c_j): \vec{y} \neq \vec{x}} \text{sim}(\vec{x}, \vec{y})$$

- Compromise between single and complete link.
- Averaged across all ordered pairs in the merged cluster instead of unordered pairs *between* the two clusters (to encourage tighter final clusters).

# Average Link Example

---



# Computing Group Average Similarity

---

- Assume cosine similarity and normalized vectors with unit length.
- Always maintain sum of vectors in each cluster.

$$\vec{s}(c_j) = \sum_{\vec{x} \in c_j} \vec{x}$$

Complete Linkage

- Compute similarity of clusters in constant time:

$$\text{sim}(c_i, c_j) = \frac{(\vec{s}(c_i) + \vec{s}(c_j)) \bullet (\vec{s}(c_i) + \vec{s}(c_j)) - (|c_i| + |c_j|)}{(|c_i| + |c_j|)(|c_i| + |c_j| - 1)}$$

# Direct Clustering (Divisive) Method

---

- *Direct clustering* methods require a specification of the number of clusters,  $k$ , desired.
- A *clustering evaluation function* assigns a real-value quality measure to a clustering.
- The number of clusters can be determined automatically by explicitly generating clusters for multiple values of  $k$  and choosing the best result according to a clustering evaluation function.

# Non-Hierarchical Clustering

---

- Typically must provide the number of desired clusters,  $k$ .
- Randomly choose  $k$  instances as *seeds*, one per cluster.
- Form initial clusters based on these seeds.
- Iterate, repeatedly reallocating instances to different clusters to improve the overall clustering.
- Stop when clustering converges or after a fixed number of iterations.

# K-Means

---

- Assumes instances are real-valued vectors.
- Clusters based on *centroids*, *center of gravity*, or mean of points in a cluster,  $c$ :

$$\vec{\mu}(c) = \frac{1}{|c|} \sum_{\vec{x} \in c} \vec{x}$$

- Reassignment of instances to clusters is based on distance to the current cluster centroids.

# Distance Metrics

---

- Euclidian distance ( $L_2$  norm):

$$L_2(\vec{x}, \vec{y}) = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

- $L_1$  norm:

$$L_1(\vec{x}, \vec{y}) = \sum_{i=1}^m |x_i - y_i|$$

- Cosine Similarity (transform to a distance by subtracting from 1):

$$1 - \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|}$$

# K-Means Algorithm

---

Let  $d$  be the distance measure between instances.

Select  $k$  random instances  $\{s_1, s_2, \dots, s_k\}$  as seeds.

Until clustering converges or other stopping criterion:

For each instance  $x_i$ :

Assign  $x_i$  to the cluster  $c_j$  such that  $d(x_i, s_j)$  is minimal.

*(Update the seeds to the centroid of each cluster)*

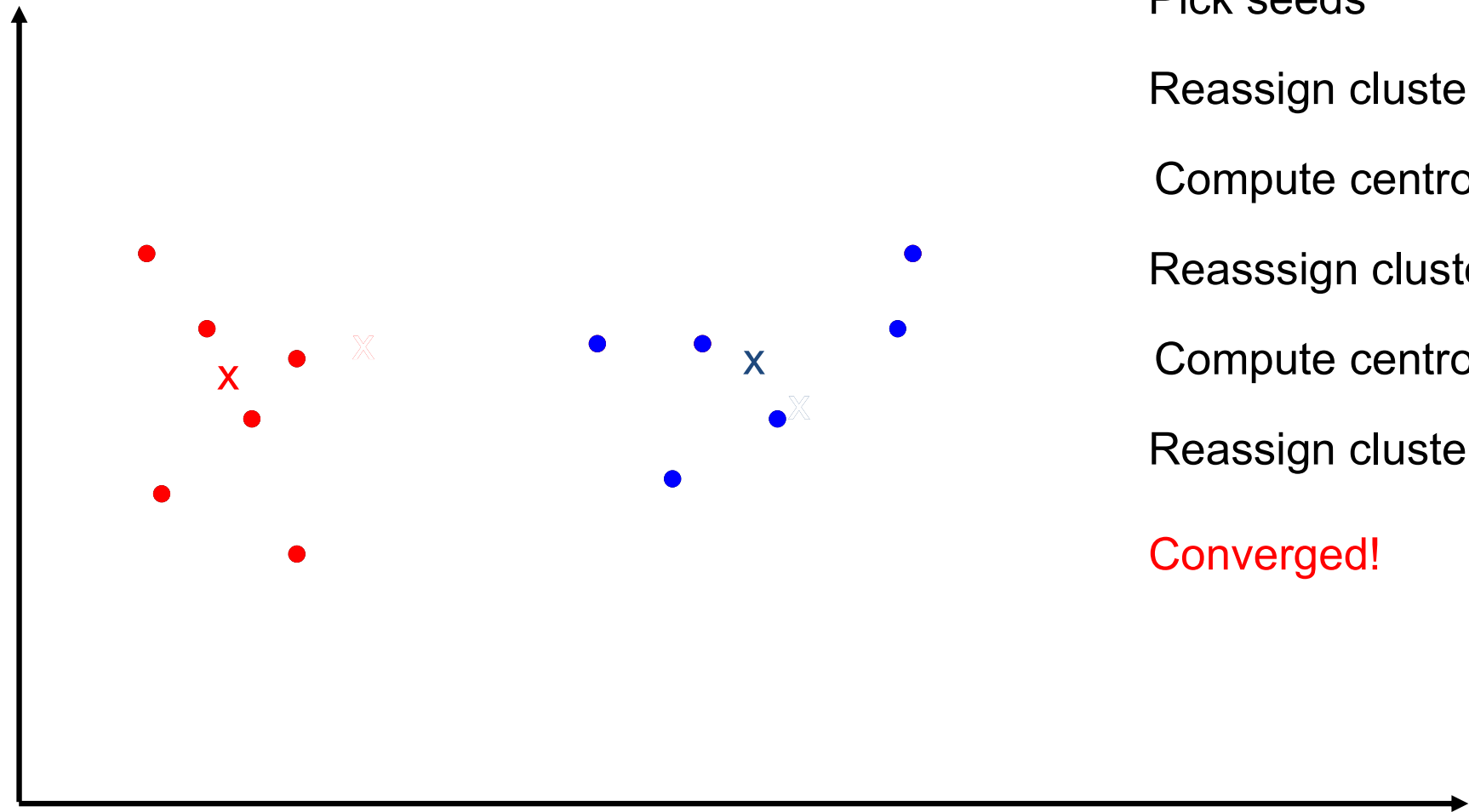
For each cluster  $c_j$

$$s_j = \mu(c_j)$$

# K-Means Example

## (K=2)

---



Pick seeds

Reassign clusters

Compute centroids

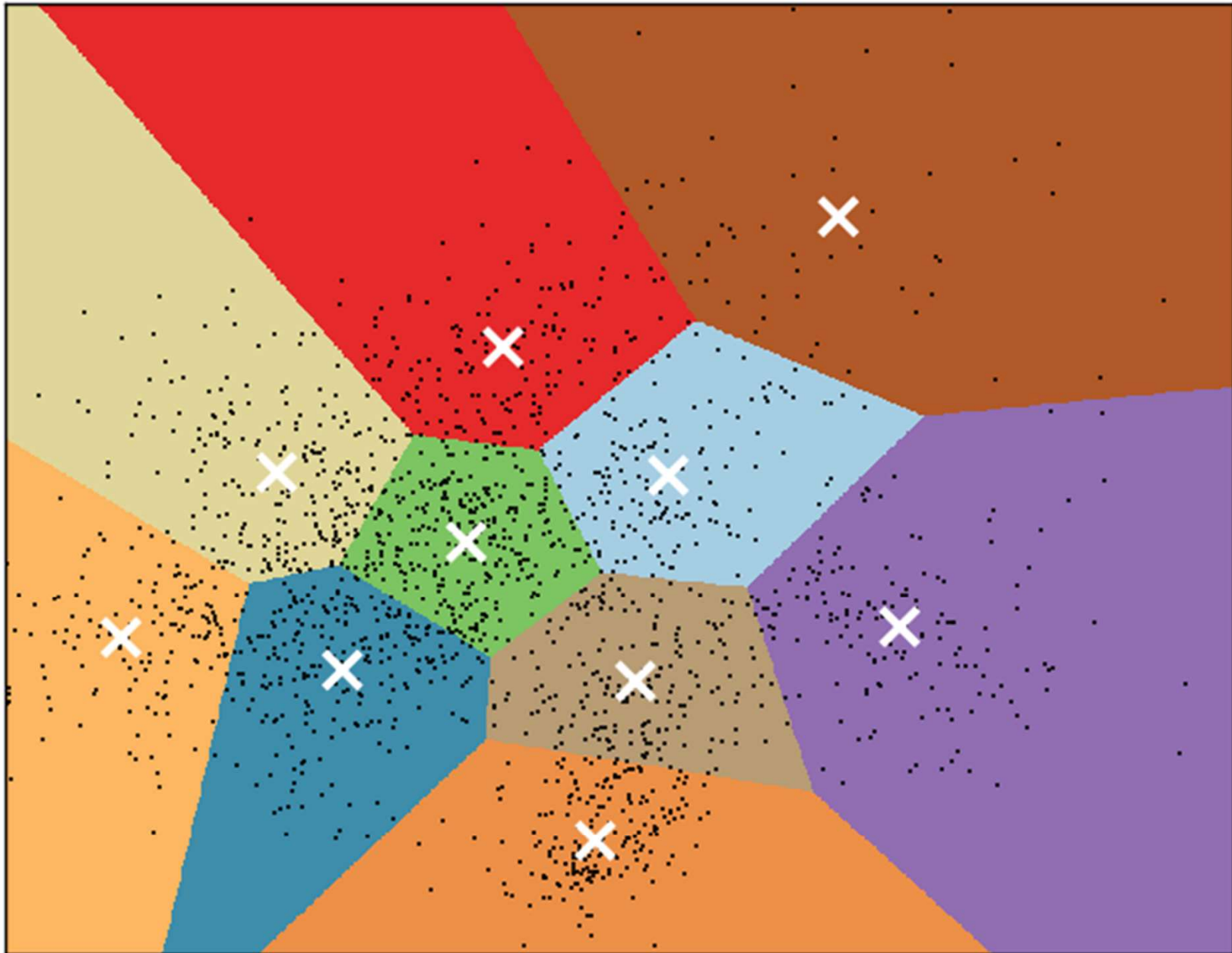
Reassign clusters

Compute centroids

Reassign clusters

**Converged!**

K-means clustering on the digits dataset (PCA-reduced data)  
Centroids are marked with white cross



# Time Complexity

---

- Assume computing distance between two instances is  $O(m)$  where  $m$  is the dimensionality of the vectors.
- Reassigning clusters:  $O(kn)$  distance computations, or  $O(knm)$ .
- Computing centroids: Each instance vector gets added once to some centroid:  $O(nm)$ .
- Assume these two steps are each done once for  $l$  iterations:  $O(lknm)$ .
- Linear in all relevant factors, assuming a fixed number of iterations, more efficient than  $O(n^2)$  HAC.

# K-Means Objective

---

- The objective of k-means is to minimize the total sum of the squared distance of every point to its corresponding cluster centroid.
- Finding the global optimum is NP-hard.
- The k-means algorithm is guaranteed to converge a local optimum.

# Seed Choice

---

- Results can vary based on random seed selection.
- Some seeds can result in poor convergence rate, or convergence to sub-optimal clusterings.
- Select good seeds using a heuristic or the results of another method.

# k-means weaknesses

---

- Initial grouping will determine the cluster significantly.
- K must be determined before hand.
- Using the same data, if it is inputted in a different order may produce different cluster if the number of data is a few.
- Weakness of arithmetic mean is not robust to outliers. Very far data from the centroid may pull the centroid away from the real one.

# k-means ++

---

- Algorithm steps
  - Randomly select a data point as the first cluster center
  - For each point  $x$  in the data set, calculate the distance  $D(x)$  from the nearest cluster center
  - Select a new data point as the new cluster center.
    - The selection principle is: the point with larger  $D(x)$  as the cluster center
  - Repeat 2 and 3 until  $k$  cluster centers are selected
  - Use these  $k$  initial cluster centers to run the standard k-means algorithm

# Buckshot Algorithm

---

- Combines HAC and K-Means clustering.
- First randomly take a sample of instances of size  $\sqrt{n}$
- Run group-average HAC on this sample, which takes only  $O(n)$  time.
- Use the results of HAC as initial seeds for K-means.
- Overall algorithm is  $O(n)$  and avoids problems of bad seed selection.

# Soft Clustering

---

- Clustering typically assumes that each instance is given a “hard” assignment to exactly one cluster.
- Does not allow uncertainty in class membership or for an instance to belong to more than one cluster.
- *Soft clustering* gives probabilities that an instance belongs to each of a set of clusters.
- Each instance is assigned a probability distribution across a set of discovered categories (probabilities of all categories must sum to 1).

# Expectation Maximization (EM)

---

- Probabilistic method for soft clustering.
- Direct method that assumes  $k$  clusters:  $\{c_1, c_2, \dots, c_k\}$
- Soft version of  $k$ -means.
- Assumes a probabilistic model of categories that allows computing  $P(c_i | E)$  for each category,  $c_i$ , for a given example,  $E$ .
- For text, typically assume a naïve-Bayes category model.
  - Parameters  $\theta = \{P(c_i), P(w_j | c_i): i \in \{1, \dots, k\}, j \in \{1, \dots, |V|\}\}$

# EM Algorithm

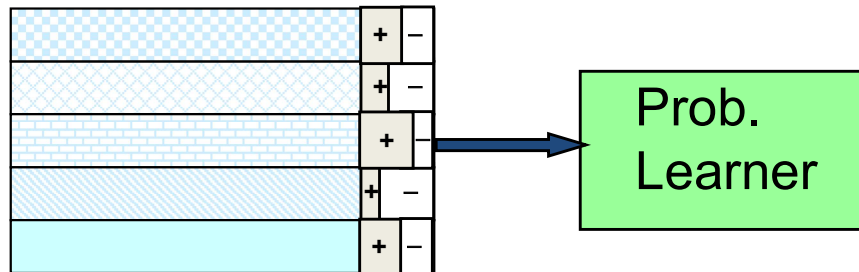
---

- Iterative method for learning probabilistic categorization model from unsupervised data.
- Initially assume random assignment of examples to categories.
- Learn an initial probabilistic model by estimating model parameters  $\theta$  from this randomly labeled data.
- Iterate following two steps until convergence:
  - **Expectation (E-step)**: Compute  $P(c_i | E)$  for each example given the current model, and probabilistically re-label the examples based on these posterior probability estimates.
  - **Maximization (M-step)**: Re-estimate the model parameters,  $\theta$ , from the probabilistically re-labeled data.

---

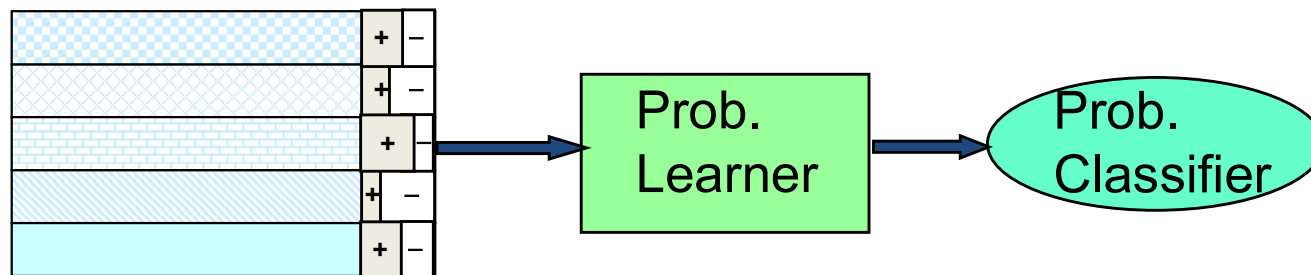
Initialize:

Give soft-labeled training data to a probabilistic learner



---

Initialize:  
Produce a probabilistic classifier



---

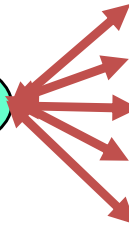
## Relabel unlabeled data using the trained classifier

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |

Prob.  
Learner



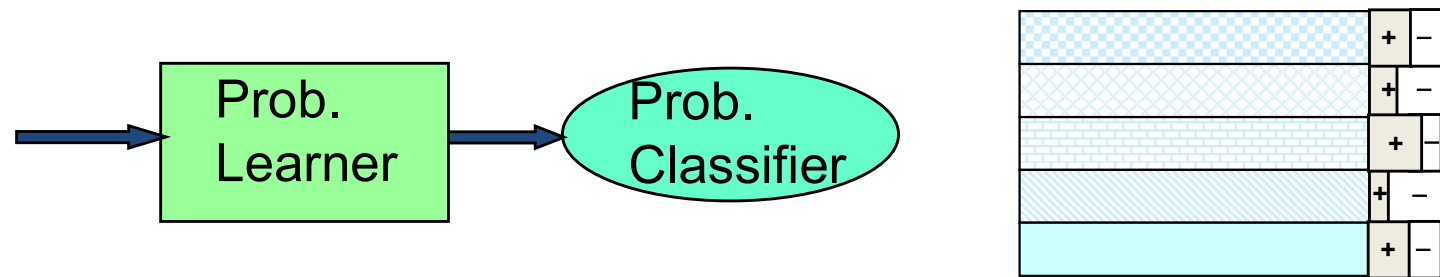
Prob.  
Classifier



|   |   |
|---|---|
| + | - |
| + | - |
| + | - |
| + | - |
| + | - |

---

Retrain classifier on relabeled data



Continue EM iterations until probabilistic labels on unlabeled data converge.

# Learning from Probabilistically Labeled Data

---

- Instead of training data labeled with “hard” category labels, training data is labeled with “soft” probabilistic category labels.
- When estimating model parameters  $\theta$  from training data, weight counts by the corresponding probability of the given category label.
- For example, if  $P(c_1 \mid E) = 0.8$  and  $P(c_2 \mid E) = 0.2$ , each word  $w_j$  in  $E$  contributes only 0.8 towards the counts  $n_1$  and  $n_{1j}$ , and 0.2 towards the counts  $n_2$  and  $n_{2j}$ .

# Naïve Bayes EM

---

Randomly assign examples probabilistic category labels.

Use standard naïve-Bayes training to learn a probabilistic model with parameters  $\theta$  from the labeled data.

Until convergence or until maximum number of iterations reached:

**E-Step:** Use the naïve Bayes model  $\theta$  to compute  $P(c_i | E)$  for each category and example, and re-label each example using these probability values as soft category labels.

**M-Step:** Use standard naïve-Bayes training to re-estimate the parameters  $\theta$  using these new probabilistic category labels.

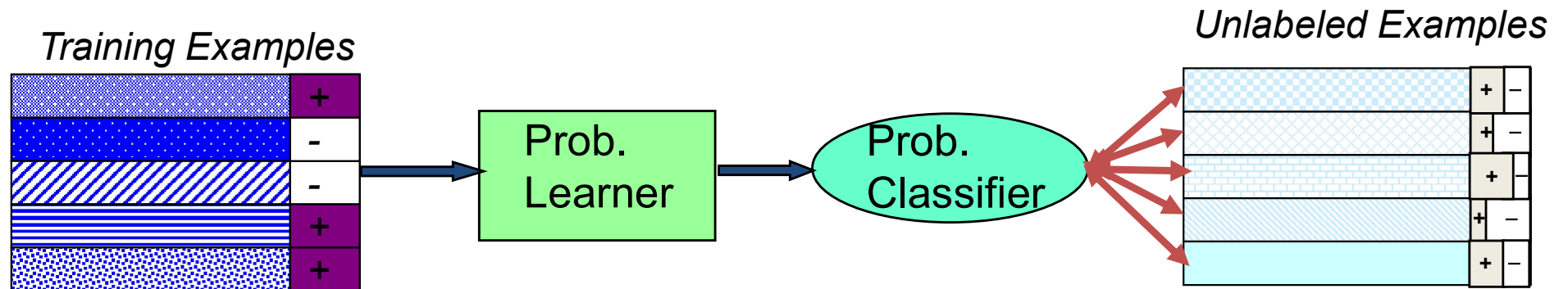
# Semi-Supervised Learning

---

- For supervised categorization, generating labeled training data is expensive.
- **Idea**: Use unlabeled data to aid supervised categorization.
- Use EM in a *semi-supervised* mode by training EM on both labeled and unlabeled data.
  - Train initial probabilistic model on user-labeled subset of data instead of randomly labeled unsupervised data.
  - Labels of user-labeled examples are “frozen” and never relabeled during EM iterations.
  - Labels of unsupervised data are constantly probabilistically relabeled by EM.

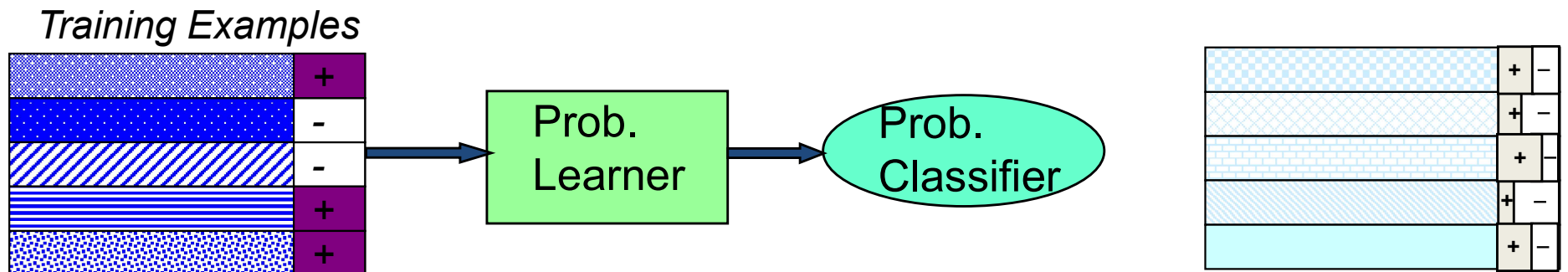
# Semi-Supervised EM

---



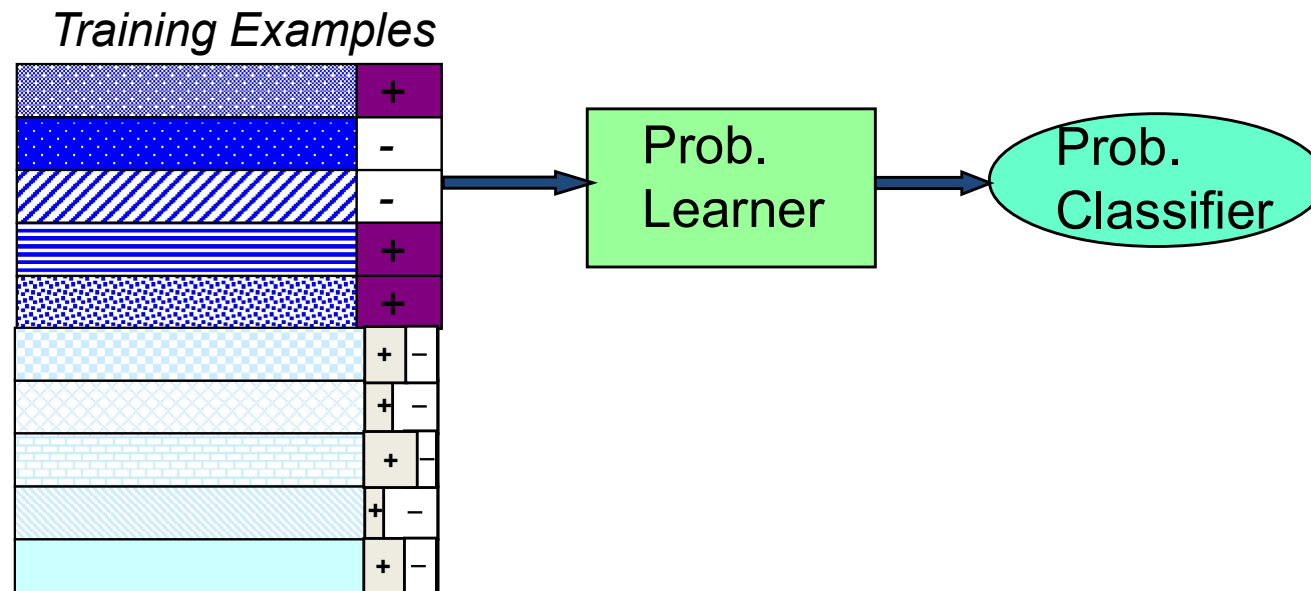
# Semi-Supervised EM

---



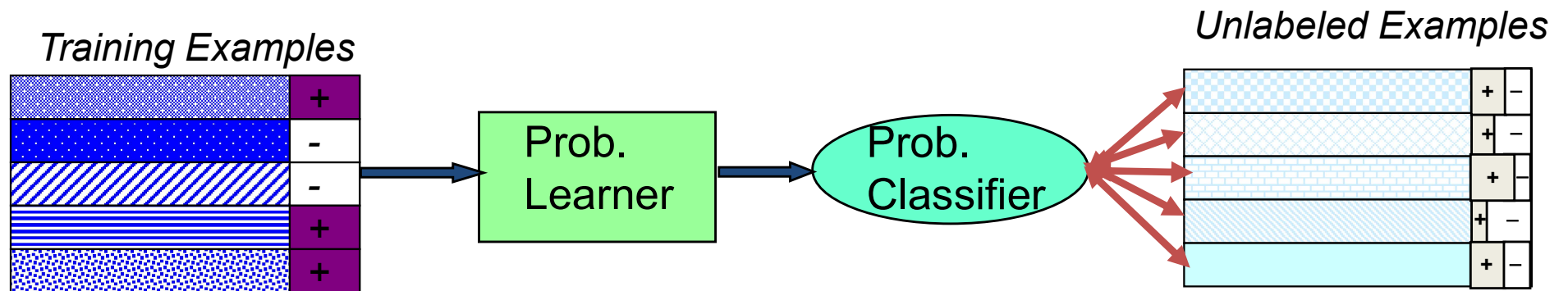
# Semi-Supervised EM

---



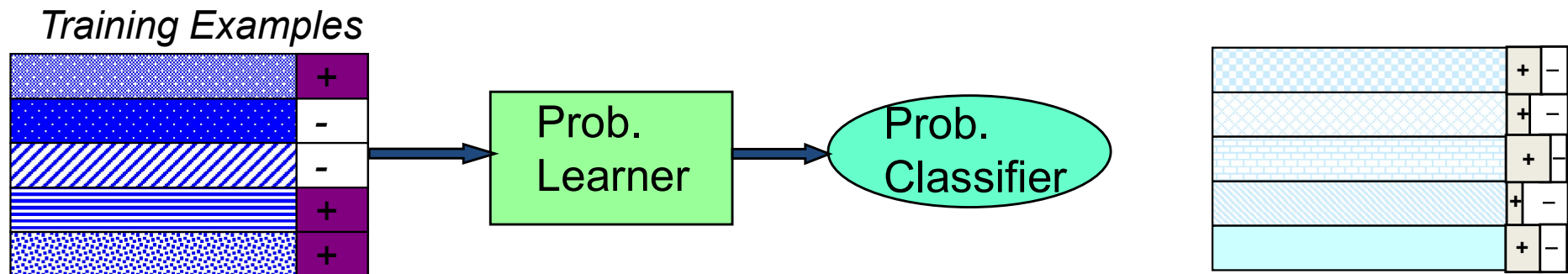
# Semi-Supervised EM

---



# Semi-Supervised EM

---



Continue retraining iterations until probabilistic labels on unlabeled data converge.

# Semi-Supervised EM Results

---

- Experiments on assigning messages from 20 Usenet newsgroups their proper newsgroup label.
- With very few labeled examples (2 examples per class), semi-supervised EM significantly improved predictive accuracy:
  - 27% with 40 labeled messages only.
  - 43% with 40 labeled + 10,000 unlabeled messages.
- With more labeled examples, semi-supervision can actually decrease accuracy, but refinements to standard EM can help prevent this.
  - Must weight labeled data appropriately more than unlabeled data.
- For semi-supervised EM to work, the “natural clustering of data” must be consistent with the desired categories
  - Failed when applied to English POS tagging (Merialdo, 1994)

# Semi-Supervised EM Example

---

- Assume “Catholic” is present in both of the labeled documents for soc.religion.christian, but “Baptist” occurs in the *labeled* data for this class.
- From labeled data, we learn that “Catholic” is highly indicative of the “Christian” category.
- When labeling unsupervised data, we label several documents with “Catholic” *and* “Baptist” correctly with the “Christian” category.
- When retraining, we learn that “Baptist” is also indicative of a “Christian” document.
- Final learned model is able to correctly assign documents containing *only* “Baptist” to “Christian”.

# Density-based Clustering

---

- The clustering structure can be determined by the tightness of the sample distribution
- Investigates the connectability between samples from the perspective of sample density
- Continuously expands clusters based on connectable samples to obtain the final clustering results

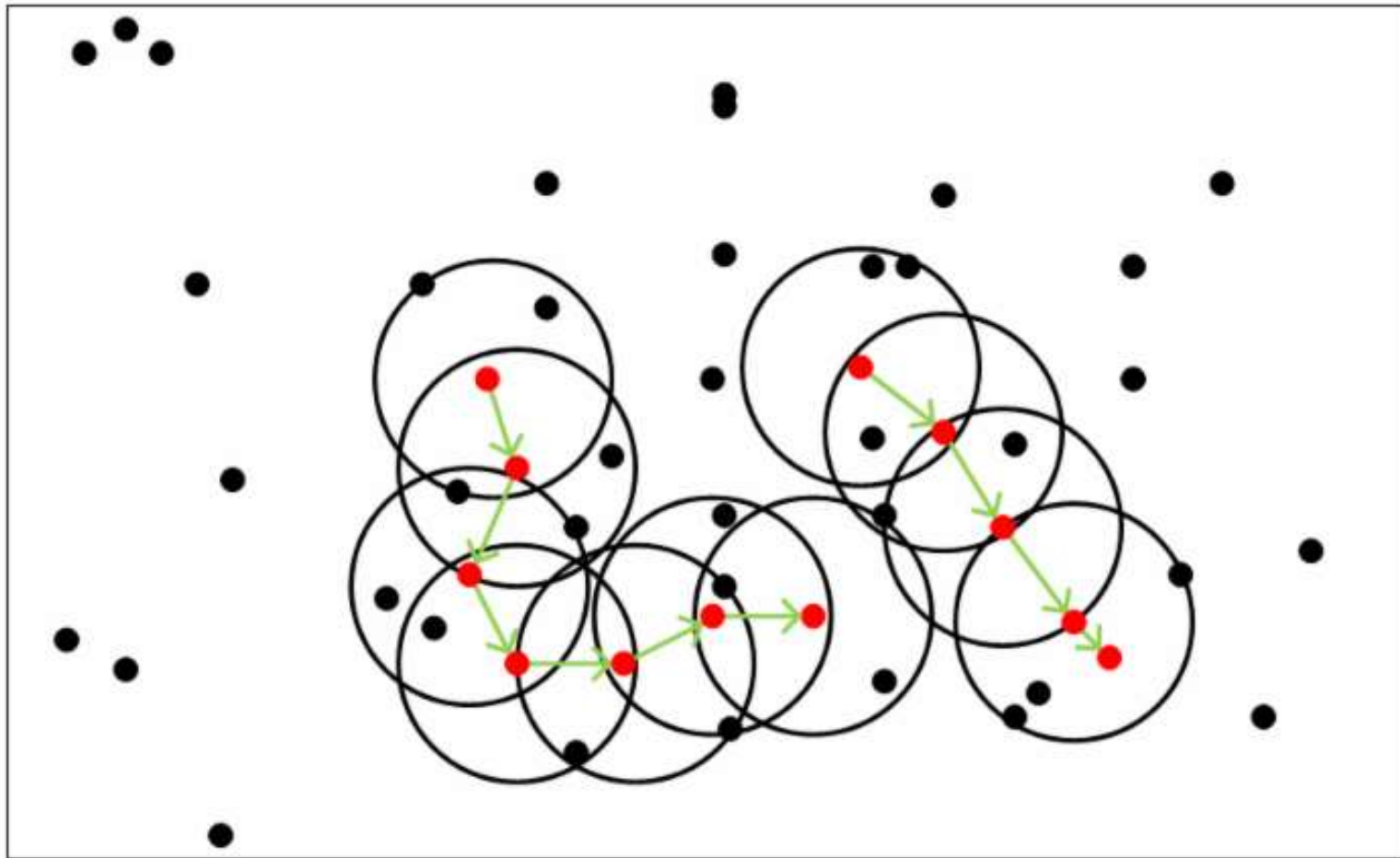
# Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

---

- Let  $\varepsilon$  be a parameter specifying the radius of a neighborhood with respect to some point. the points are classified as core points, (density-) reachable points and outliers, as follows:
  - A point  $p$  is a **core point** if at least  $minPts$  points are within distance  $\varepsilon$  of it
  - A point  $q$  is **directly reachable** from  $p$  if point  $q$  is within distance  $\varepsilon$  from core point  $p$
  - A point  $q$  is **reachable** from  $p$  if there is a path  $p_1, \dots, p_n$  with  $p_1 = p$  and  $p_n = q$
  - All points **not reachable** from any other point are outliers or noise points

# Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

---



# Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

---

- Abstract algorithm
  - ① selects a core object without a category as the seed, and then finds all the sample sets with the density of this core object that can be reached.
  - ② select another core object with no category to find a set of samples with reachable density, so that another cluster is obtained.
  - ③ Assign each non-core point to a nearby cluster if the cluster is an  $\epsilon$  (eps) neighbor, otherwise assign it to noise.

# Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

---

- Complexity
  - if an indexing structure is, an overall average runtime complexity is  $O(n \log n)$
  - without the use of an accelerating index structure, or on degenerated data, the worst case run time complexity remains  $O(n^2)$

# Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

---

- Pros
  - does not require one to specify the number of clusters in the data a priori
  - has a notion of noise, and is robust to outliers
  - the clustering results are not biased
- Cons
  - the quality of clustering is affected by the density of the sample set
  - for large data sets, the convergence time is longer
  - parameter tuning is tricky

# Issues in Clustering

---

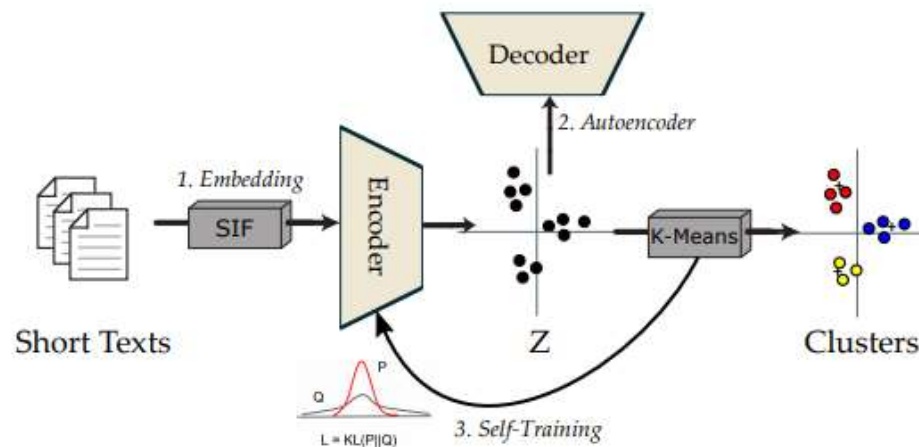
- How to evaluate clustering?
  - Internal:
    - Tightness and separation of clusters (e.g. k-means objective)
    - Fit of probabilistic model to data
  - External
    - Compare to known class labels on benchmark data
- Improving search to converge faster and avoid local minima.
- Overlapping clustering.

# A Self-Training Approach for Short Text Clustering

**(Amir Hadifar et.al., ACL 2019)**

---

- Short text clustering
- Traditional bag-of-words or TFIDF representations lead to sparse vector representations for short texts.
- Low dimensional continuous representations or embeddings can counter that sparseness problem:
- Learns discriminative features from both an autoencoder and a sentence embedding, then uses assignments from a clustering algorithm as supervision to update weights of the encoder network.



# Unsupervised Domain Clusters in Pretrained Language Models

*(Roei Aharoni et.al., EMNLP 2020)*

---

- Pre-trained language models implicitly learn sentence representations that cluster by domains without supervision

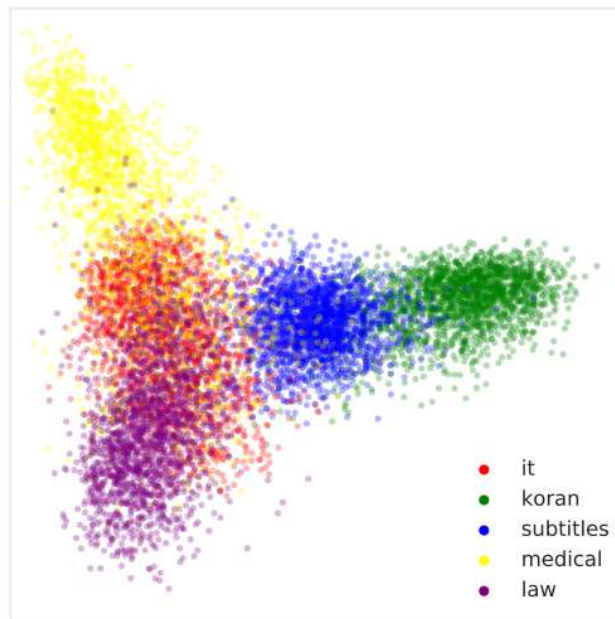


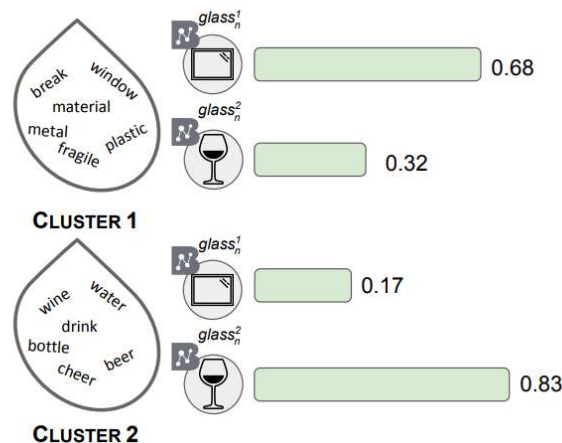
Figure 1: A 2D visualization of average-pooled BERT hidden-state sentence representations using PCA. The colors represent the domain for each sentence.

# CluBERT: A Cluster-Based Approach for Learning Sense Distributions in Multiple Languages

(*Tommaso Pasini et.al., ACL 2020*)

---

- Word Sense Disambiguation (WSD)
- Exploite the assumption that similar meanings appear in similar contexts and the representational power of BERT
- CluBERT learns distributions over English senses that are of higher quality than those extracted by alternative approaches

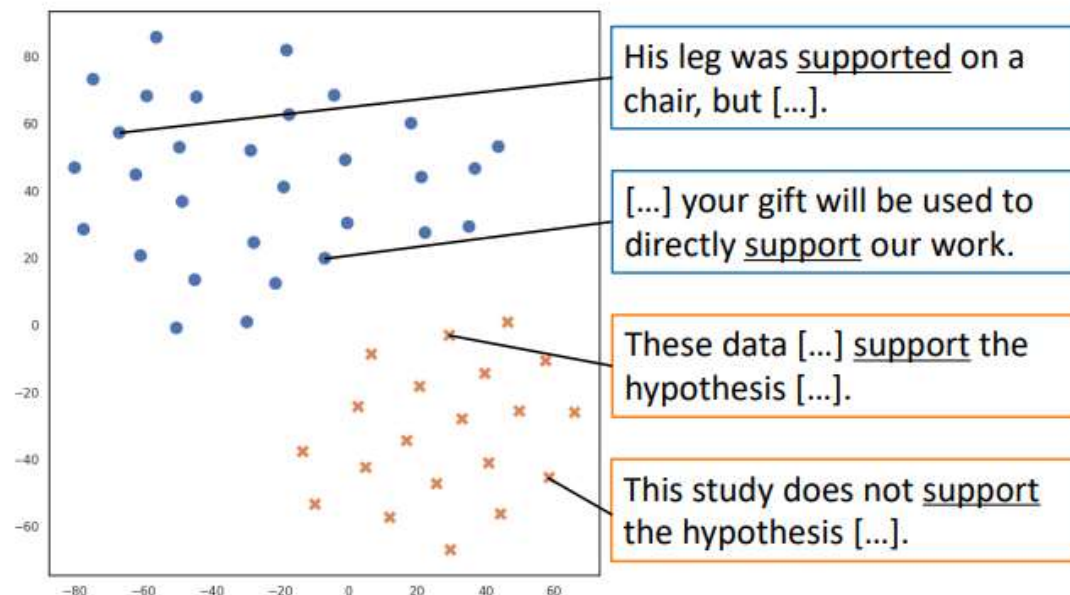


# Verb Sense Clustering using Contextualized Word Representations for Semantic Frame Induction

*(Kosuke Yamada et.al., ACL 2021)*

---

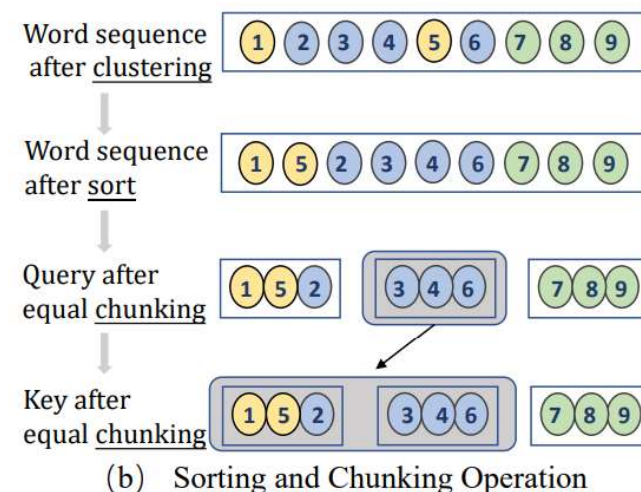
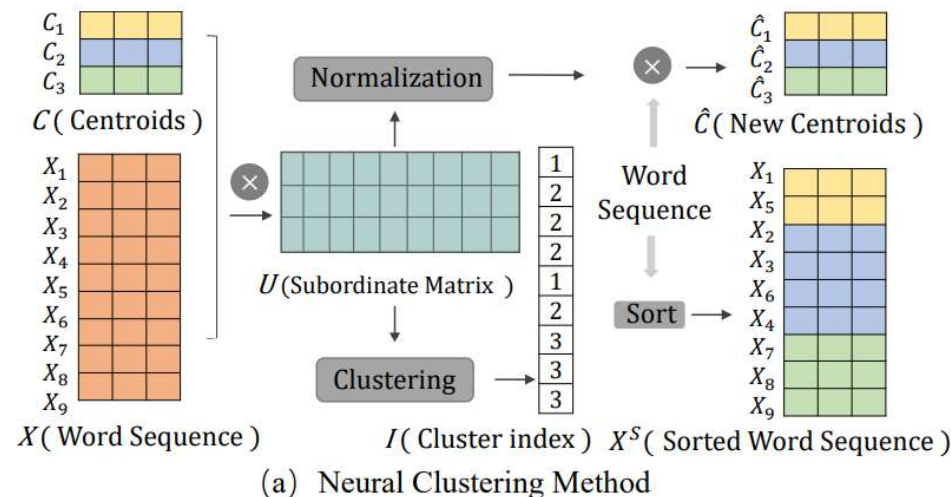
- how well contextualized word representations can recognize the difference of frames that the same verb evokes
- explore which types of representation are suitable for semantic frame induction



# ClusterFormer: Neural Clustering Attention for Efficient and Effective Transformer

(Ningning Wang *et.al.*, ACL 2022)

- Design a neural clustering method, which can be seamlessly integrated into the Self-Attention Mechanism in Transformer.
- The clustering task and the target task are jointly trained and optimized to benefit each other, leading to significant effectiveness improvement.



# Topic Model

---

- Discover hidden topical patterns that pervade the collection through statistical regularities
- Annotate documents with these topics
- Use the topic annotations to organize, summarize, search texts ...



# Topic Examples

---

- An illustration of four (out of 300) topics extracted from TASA corpus

Topic 247

| word      | prob. |
|-----------|-------|
| DRUGS     | .069  |
| DRUG      | .060  |
| MEDICINE  | .027  |
| EFFECTS   | .026  |
| BODY      | .023  |
| MEDICINES | .019  |
| PAIN      | .016  |
| PERSON    | .016  |
| MARIJUANA | .014  |
| LABEL     | .012  |
| ALCOHOL   | .012  |
| DANGEROUS | .011  |
| ABUSE     | .009  |
| EFFECT    | .009  |
| KNOWN     | .008  |
| PILLS     | .008  |

Topic 5

| word    | prob. |
|---------|-------|
| RED     | .202  |
| BLUE    | .099  |
| GREEN   | .096  |
| YELLOW  | .073  |
| WHITE   | .048  |
| COLOR   | .048  |
| BRIGHT  | .030  |
| COLORS  | .029  |
| ORANGE  | .027  |
| BROWN   | .027  |
| PINK    | .017  |
| LOOK    | .017  |
| BLACK   | .016  |
| PURPLE  | .015  |
| CROSS   | .011  |
| COLORED | .009  |

Topic 43

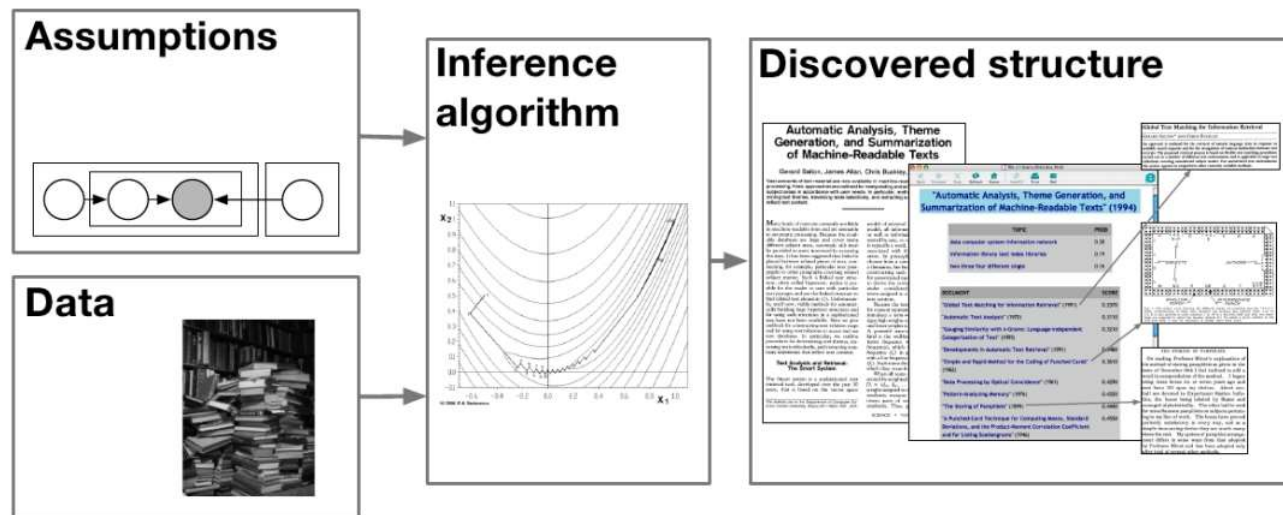
| word       | prob. |
|------------|-------|
| MIND       | .081  |
| THOUGHT    | .066  |
| REMEMBER   | .064  |
| MEMORY     | .037  |
| THINKING   | .030  |
| PROFESSOR  | .028  |
| FELT       | .025  |
| REMEMBERED | .022  |
| THOUGHTS   | .020  |
| FORGOTTEN  | .020  |
| MOMENT     | .020  |
| THINK      | .019  |
| THING      | .016  |
| WONDER     | .014  |
| FORGET     | .012  |
| RECALL     | .012  |

Topic 56

| word      | prob. |
|-----------|-------|
| DOCTOR    | .074  |
| DR.       | .063  |
| PATIENT   | .061  |
| HOSPITAL  | .049  |
| CARE      | .046  |
| MEDICAL   | .042  |
| NURSE     | .031  |
| PATIENTS  | .029  |
| DOCTORS   | .028  |
| HEALTH    | .025  |
| MEDICINE  | .017  |
| NURSING   | .017  |
| DENTAL    | .015  |
| NURSES    | .013  |
| PHYSICIAN | .012  |
| HOSPITALS | .011  |

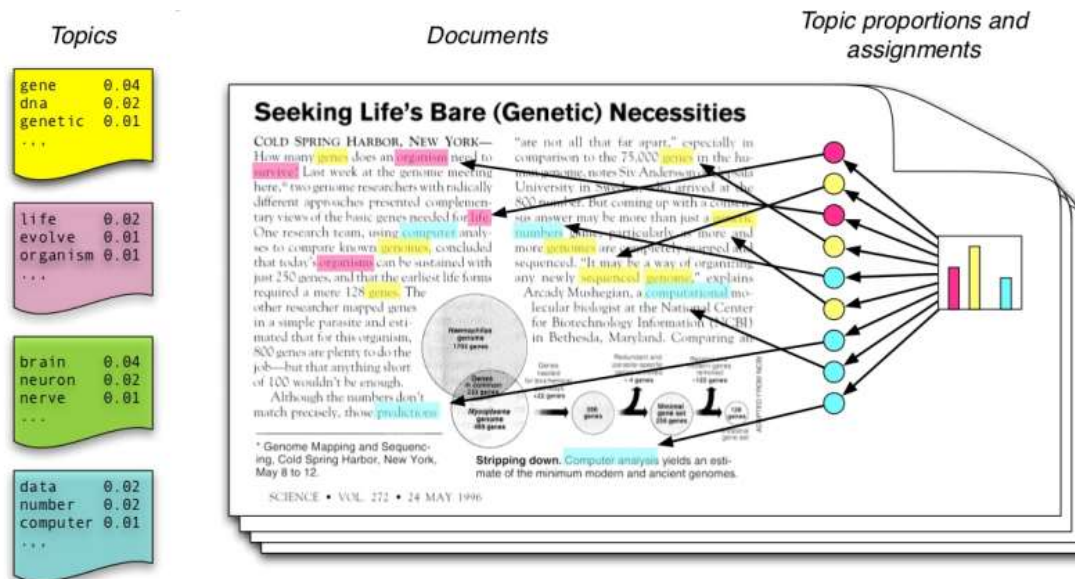
# Topic model : LDA

- Observation: a collection of texts
- Assumption: the texts have been generated according to some model
- Output: the model that has generated the texts



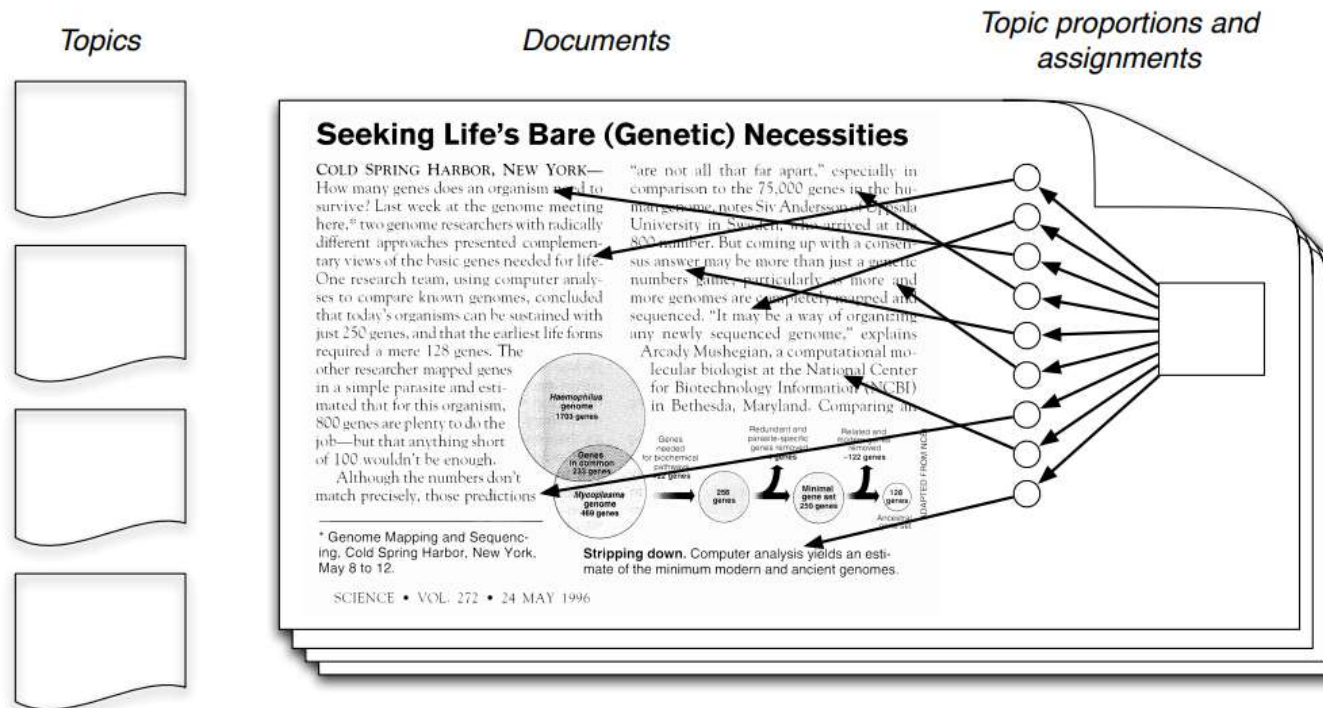
# Topic model : LDA

- Each **topic** is a distribution over words
- Each **document** is a mixture of corpus-wide topics
- Each **word** is drawn from one of these topics
- We only observe the words within the documents and the other structure are **hidden variables**.



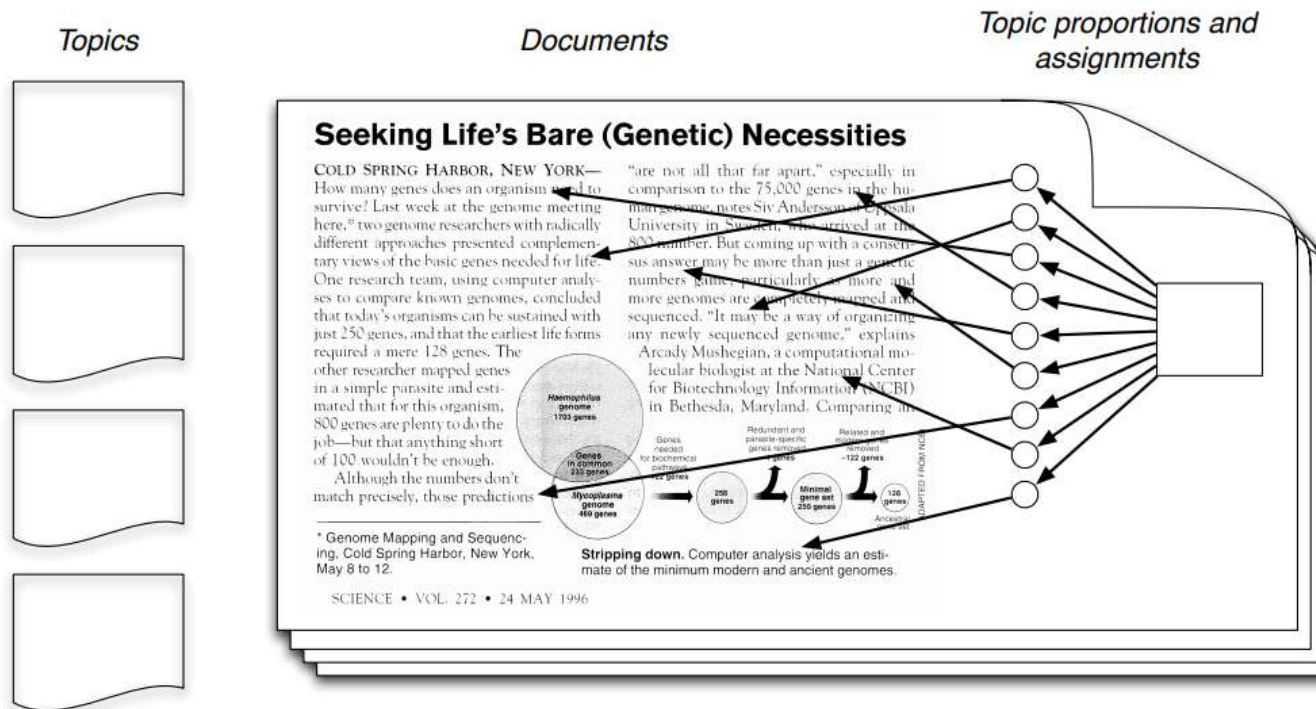
# Topic model : LDA

- In reality, we only observe the documents
- The other structure are ***hidden variables***



# Topic model : LDA

- Our goal is to *infer* the hidden variables
- I.e., compute their distribution conditioned on the documents  
 ***$p(\text{topics, proportions, assignments} \mid \text{documents})$***



# Setting up a generative model

---

- We have  $D$  documents using a vocabulary of  $V$  word types
- Each documents contains (up to)  $N$  word tokens
- We assume  $K$  topics
- Each document has a  $K$  dimensional multinomial  $\theta_d$  over topics with a common Dirichlet prior  $Dir(\alpha)$
- Each topic has a  $V$  dimensional multinomial  $\beta_k$  over words with a common symmetric Dirichlet prior  $D(\eta)$

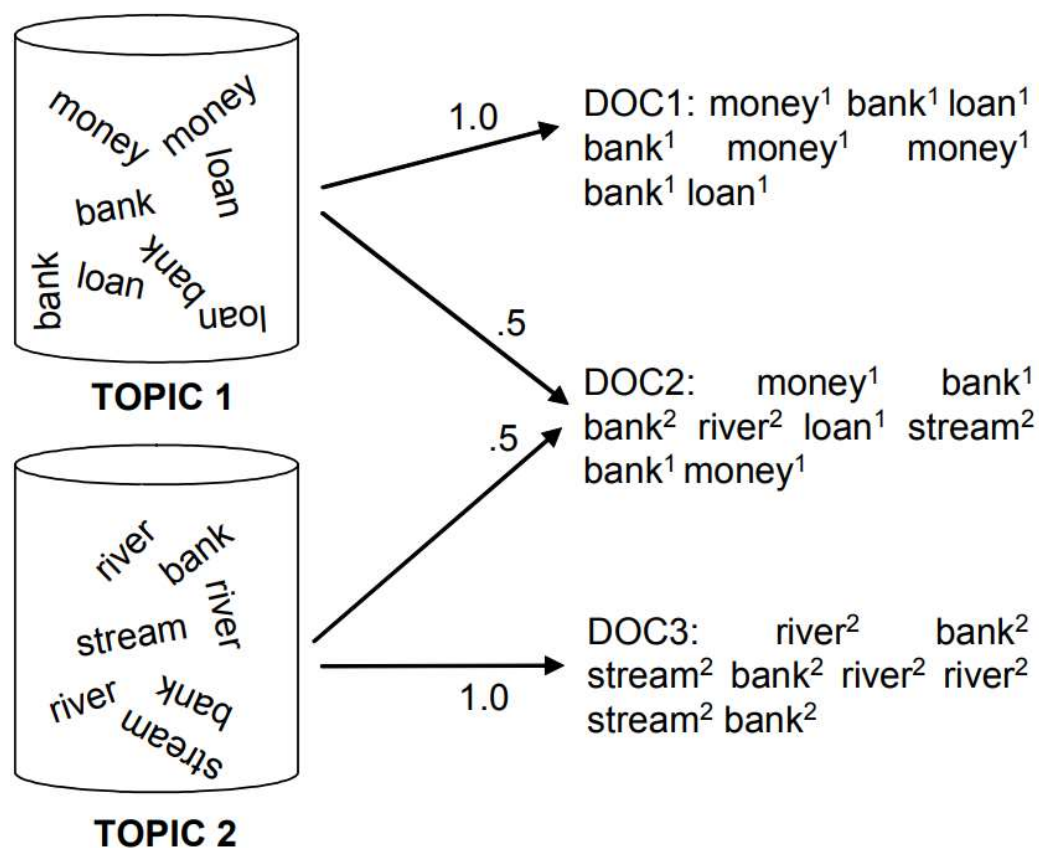
# The generative process

---

- For each **topic 1...k**:
  - Draw a **multinomial over words**  $\beta_k \propto D(\eta)$
- For each **document 1...d**:
  - Draw a **multinomial over topics**  $\beta_k \propto D(\alpha)$
  - For each word  $w_{d,n}$  :
    - Draw a topic  $Z_{d,n} \propto \text{Mult}(\theta_d)$  with  $Z_{d,n} \in [1...K]$
    - Draw a word  $w_{d,n} \propto \text{Mult}(\beta_{Z_{d,n}})$

# The generative process

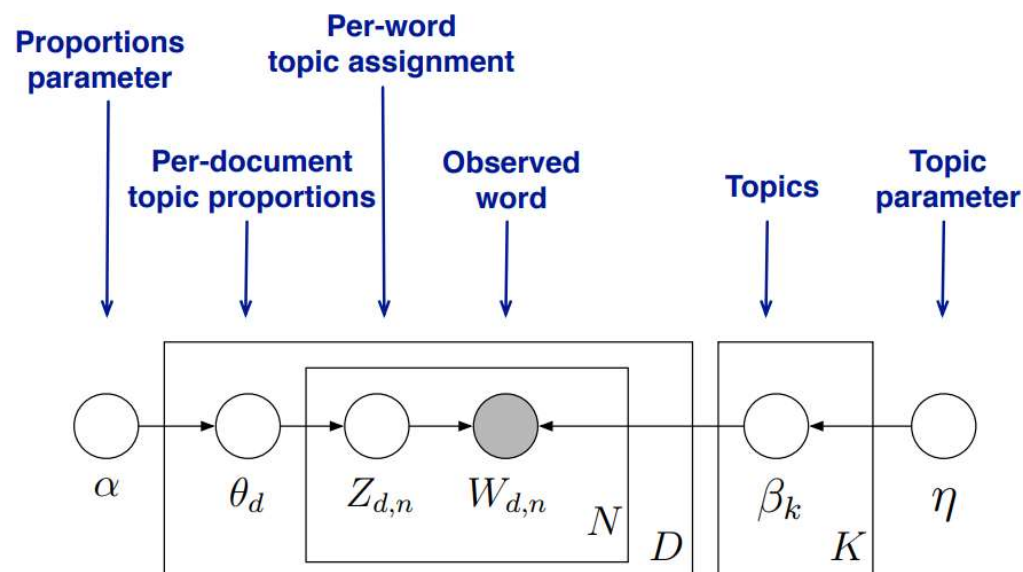
---



# LDA as a graphical model

---

- Nodes are random variables; edges indicate dependence
- Shaded nodes are observed
- Plates indicate replicated variables



# The inference problem

---

- What is the ***posterior of the hidden variables*** given the observed variables (and hyperparameters)?

$$p(\vec{\theta}_{1:D}, z_{1:D, 1:N}, \vec{\beta}_{1:K} \mid w_{1:D, 1:N}, \alpha, \eta) = \frac{p(\vec{\theta}_{1:D}, \vec{z}_{1:D}, \vec{\beta}_{1:K} ; \vec{w}_{1:D}, \alpha, \eta)}{\int_{\vec{\beta}_{1:K}} \int_{\vec{\theta}_{1:D}} \sum_{\vec{z}} p(\vec{\theta}_{1:D}, \vec{z}_{1:D}, \vec{\beta}_{1:K} ; \vec{w}_{1:D}, \alpha, \eta)}.$$

- Problem: the integral in the denominator is ***intractable***
- Solutions : approximate inference
  - Gibbs sampling
  - Variational Inference

# Gibbs Sampling

---

- Gibbs sampling is an example of a **Markov Chain Monte Carlo** (MCMC) technique
- Construct a **Markov chain** on the hidden variables, whose limiting distribution is the posterior.
- Collect **independent samples** from that distribution; approximate the posterior with them
- The MCMC method is used to estimate the posterior distribution of the parameters of interest through **random sampling** in the probability space

# Variational Inference

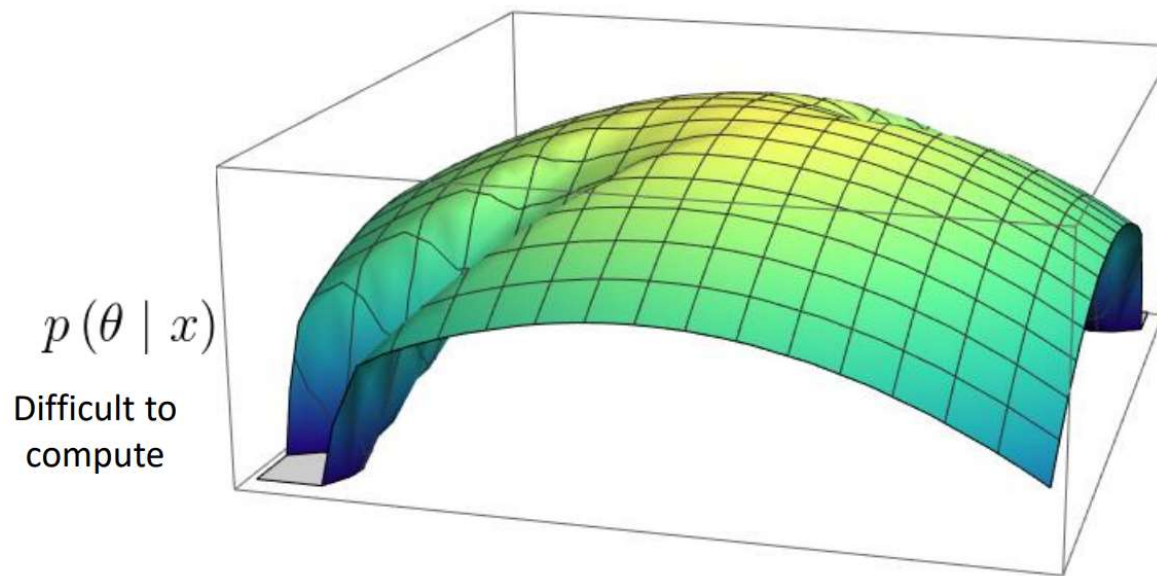
---

- Variational inference replaces sampling with ***optimization***.
- The main idea is :
  - Place a distribution over the hidden variables with free parameters, called ***variational parameters***.
  - Optimize the variational parameters to make the distribution close (in KL divergence) to the true posterior
- In some settings, variational inference is faster than MCMC

# Variational Inference

---

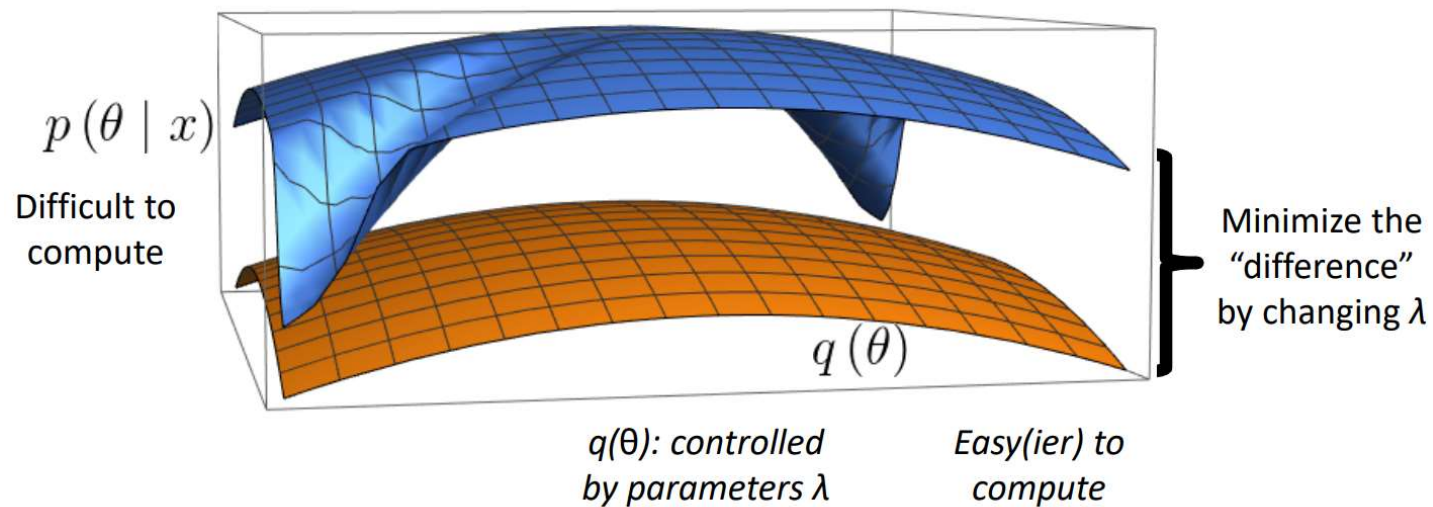
- Can use neural networks to approximate the *posterior distribution*



# Variational Inference

---

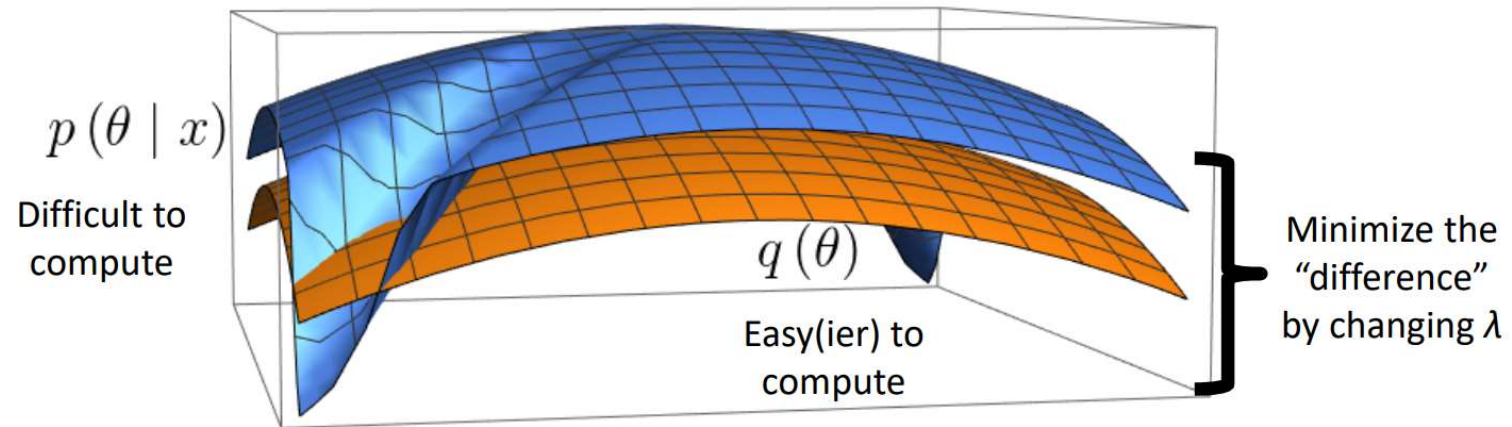
- Can use neural networks to approximate the ***posterior distribution***



# Variational Inference

---

- Can use neural networks to approximate the ***posterior distribution***



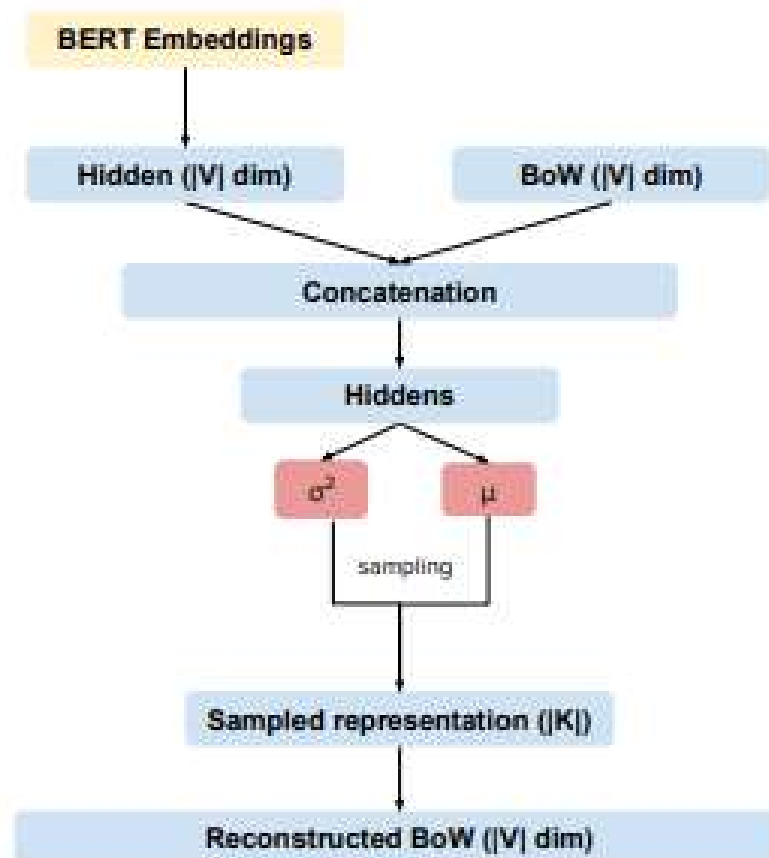
# Pre-training is a Hot Topic:

## Contextualized Document Embeddings Improve Topic Coherence

(*Federico Bianchi et.al., Arxiv 2020*)

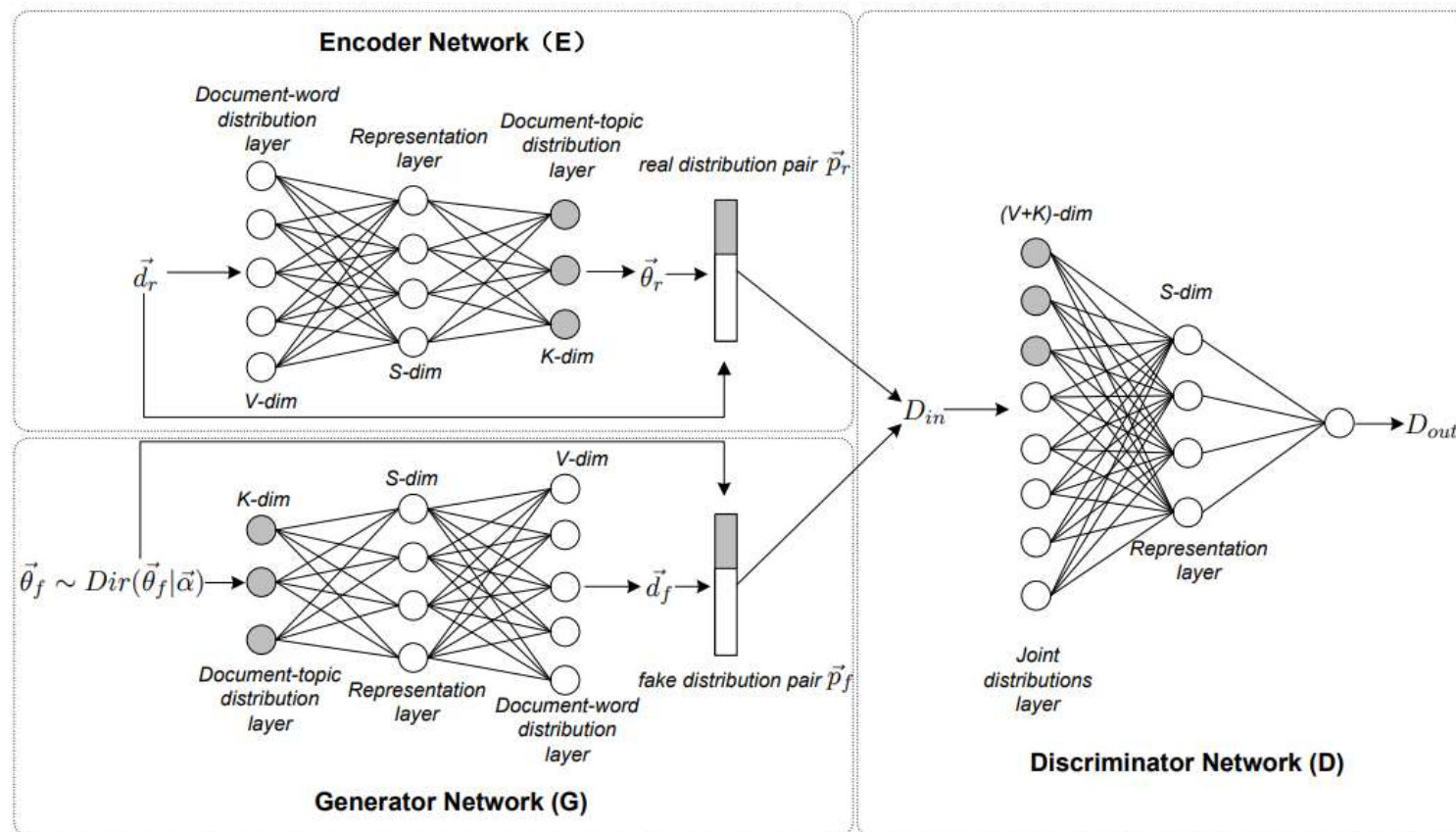
---

- Use BERT to add more contextual knowledge to the model.



# Neural Topic Modeling with Bidirectional Adversarial Training (*Rui Wang et.al., EMNLP 2020*)

- Use Adversarial Training to directly approximate Dirichlet distribution



# The Next Lecture

---

- Lecture 12  
Information Extraction